Riftbound TCG privé

Rapport d'avancement — Étape 4 : Catalogue des cartes

08/11/2026

Pour reprendre le projet dans une nouvelle conversation, donner ce PDF (ou le dépôt GitHub) à l'assistant et lui demander de lire **documentation/etat-du-projet.md** puis de continuer à l'étape indiquée comme « prochaine étape ».

Prompt initial (verbatim)

Date : 2026-11-08 — Auteur : Sébastien Girard

> Je souhaite créer un tcg sur Rifbound mais de maniere privé géré par compte et pwd quand tu accèdes au site.
>
> Je souhaite utiliser cet api https://github.com/LouisCourrian/riftbound-cards ce qui me permet de rien stocker dans ma db.
>
> Ce sera hébérger sur cloudeflare gratuitement avec sqlite de cloudflare gratuit.
>
> Il faut pouvoir choisir le pack qu'on veut ouvrir (si c'est vendetta, unleashed ect). pouvoir échanger entre les joueurs et avoir 1 packs toutes les 10 mins. on peut en stocké 15. Avoir des raretés comme dans le vrai jeu de cartes avec les memes taux de drop les memes nbr de cartes par boosters ect.
>
> Faut que ce soit responsible sur tel pc tablette ect. Qu'on puisse utiliser la fct application sur navigateur comme ça c'est plus simple sur pc.
>
> On fait étape par étape. Dans un process de devsecops.
>
> On va utiliser GIthub ça va me permettre de switch sur mon pc portable. Je veux des rapports pdf d'avancement avec le prompt de base sur le fichier comme ça si je perds notre conv tu pourras pile reprendre ou on en est sans etre perdu.

Riftbound TCG privé — État du projet

Ce fichier est la source de vérité pour reprendre le projet. Le rapport PDF de chaque étape est généré depuis ce fichier et depuis `documentation/demande-initiale.md`.

Dernière mise à jour : 2026-11-08

Étape en cours : 4 — Catalogue des cartes (terminée)

Prochaine étape : 5 — Moteur de boosters (taux de drop, tirage côté serveur, tests statistiques) et timer 1 booster / 10 min, stock max 15

Dépôt : https://github.com/Crousti-Bats/riftbound-tcg-prive (public, `main` protégée)

Site en production : https://riftbound-tcg.axelgirard.workers.dev

Infrastructure Cloudflare (aucune de ces valeurs n'est secrète)

Élément	Valeur
Compte	`Axelgirard@gmail.com's Account` (ID `14bd3e99e009ec33efebde236223ab6a`)
Worker	`riftbound-tcg` (site + API), URL `riftbound-tcg.axelgirard.workers.dev`
Base D1	`riftbound-tcg`, ID `efbe0899-7a3f-4f63-aad4-4bd9c540eb2d`, région WEUR, nom dans le code `BASE_DE_DONNEES`
Migrations	`base-de-donnees/migrations/NNNN_description.sql`, appliquées automatiquement au déploiement
GitHub	environnement `production` (déploiement depuis `main` uniquement) : secrets `CLOUDFLARE_API_TOKEN` et `CLOUDFLARE_ACCOUNT_ID`, variable `CLOUDFLARE_SOUS_DOMAINE=axelgirard`

0. Reprendre sur un autre PC

```
git clone https://github.com/Crousti-Bats/riftbound-tcg-prive.git
cd riftbound-tcg-prive
npm ci
npm run dev        # site + API en local (http://localhost:5173)
npm test           # tests
```

Pré-requis : Git, Node.js 22 ou plus, GitHub CLI (`gh auth login`), Python avec `pip install reportlab` pour les rapports.

Méthode de travail : `git switch -c etape-N`, puis commit, puis `git push`, puis `gh pr create`. On merge quand la CI est verte.

1. Objectif

Un site privé de collection et d'échange de cartes Riftbound, accessible uniquement avec un compte et un mot de passe, hébergé gratuitement sur Cloudflare.

- Ouvrir des boosters en choisissant le set (Origins, Spiritforged, Unleashed, Vendetta, Radiance).
- Composition des boosters et taux de drop alignés sur le vrai jeu.
- Recevoir 1 booster toutes les 10 minutes, avec un stock de 15 au maximum.
- Échanger des cartes entre joueurs.
- Site responsive (téléphone, tablette, PC) et installable comme application (PWA).

2. Décisions d'architecture

Sujet	Décision	Raison
Hébergement	Un seul Cloudflare Worker qui sert le front (Static Assets) et l'API	Offre gratuite, un seul déploiement, aucun CORS
Base de données	Cloudflare D1 (SQLite), offre gratuite	Demandé. Limites : 5 Go, 5 M lectures/jour, 100 k écritures/jour
Données des cartes	`cards.json` de la dernière release de LouisCourrian/riftbound-cards, récupéré par le Worker et mis en cache (Cache API, 24 h)	Aucune carte stockée en base. D1 ne garde que des `cardCode` et des quantités
Images	Liens directs vers le CDN Riot (`imageUrl`)	Rien à héberger
Backend	TypeScript + Hono	Léger, prévu pour Workers
Frontend	React + Vite + TypeScript, `vite-plugin-pwa`	PWA installable, responsive
Authentification	Inscription uniquement sur invitation : l'admin génère des codes. Mots de passe hachés en PBKDF2-SHA256 (WebCrypto), sessions en base avec cookie `HttpOnly; Secure; SameSite=Strict`	Site privé, sans dépendance externe
Aléatoire	`crypto.getRandomValues` côté serveur uniquement	Le client ne peut pas tricher
Timer des boosters	Calcul paresseux : `stock = min(15, stock_saved + floor((now - last_tick)/600 s))`, sans cron	Aucun coût, résultat exact
Échanges	Proposition, puis acceptation. Transfert atomique via `D1 batch()` après vérification de possession	Pas de duplication de cartes
Code source	GitHub (dépôt public, voir §5), synchronisé entre le PC fixe et le portable	Demandé

2bis. Espace admin et monitoring (demandé le 2026-11-08)

Une page `/admin` accessible **uniquement par l'utilisateur** (le propriétaire du site), pour surveiller le site en quasi temps réel.

Contrôle d'accès (défense en profondeur)

- Colonne `role` dans la table des comptes (`joueur` ou `admin`). Le premier compte admin est créé par une commande en ligne réservée à l'utilisateur. Aucun formulaire public ne peut créer d'admin.
- Toutes les routes `/api/admin/*` vérifient le rôle **côté serveur**, à chaque requête. Masquer le menu dans l'interface ne suffit pas.
- Option recommandée à l'étape 10 : protéger aussi `/admin` et `/api/admin/*` avec Cloudflare Access (Zero Trust, gratuit jusqu'à 50 utilisateurs). Il faudrait alors une vérification par e-mail ou par code en plus du mot de passe.
- Chaque action admin est journalisée dans la table `journal_admin` (qui, quoi, quand).

Ce que la page affiche (rafraîchie toutes les 5 à 10 secondes)

- Joueurs connectés récemment (dernière activité de moins de 5 minutes), nombre total de comptes, invitations en attente.
- Activité : boosters ouverts (dernière heure et aujourd'hui, par set), échanges proposés, acceptés ou refusés, dernières cartes rares tirées.
- Sécurité : connexions échouées, comptes ou IP bloqués par la limitation des tentatives, dernières erreurs serveur.

- Santé : état de l'API et de la base, temps de réponse, âge du cache `cards.json`, version déployée.
- Quotas gratuits Cloudflare : requêtes Worker et lectures/écritures D1 du jour, comparées aux limites. Ces chiffres viennent de l'API GraphQL Analytics de Cloudflare, avec un jeton en lecture seule stocké en `wrangler secret`, jamais dans le code.
- Actions : créer ou révoquer une invitation, désactiver un compte, forcer la déconnexion d'un joueur.

Technique : pour démarrer, le navigateur interroge le serveur toutes les quelques secondes (« polling »). C'est simple et ça tient dans l'offre gratuite. On pourra passer à du vrai temps réel (WebSocket via Durable Objects) si le besoin se confirme.

2ter. Authentification (étape 3)

Sujet	Choix
Inscription	Uniquement avec un code d'invitation (256 bits, stocké en SHA-256, usage unique, expiration de 24 h à 7 jours, pseudo réservable). Le lien `/inscription#code=…` place le code après le `#`, qui n'est jamais envoyé au serveur
Admin	Une seule façon d'en créer un : `npm run admin:invitation -- --pseudo X --production`, qui demande d'être connecté à Cloudflare. Le site ne peut créer que des invitations `joueur`. L'utilisateur a un seul compte `Crousti` (joueur et admin)
Accès admin	Rôle vérifié côté serveur à chaque requête, avec re-saisie du mot de passe (élévation) valable 15 minutes glissantes et un bouton « Fermer l'accès admin »
Mots de passe	6 à 128 caractères (minimum abaissé de 12 à 6 à la demande de l'utilisateur le 2026-11-08 ; risque accepté : un mot de passe court est vite cassé si la base fuit, mais la limitation des tentatives protège le site), PBKDF2-SHA256 à 100 000 itérations (maximum Workers), sel aléatoire, format versionné. **À surveiller** : le plan gratuit limite le CPU à 10 ms par requête, mais Cloudflare tolère des pics. Si les connexions échouent en production (erreur 1102), baisser les itérations (le format le permet)
Sessions	Jeton de 256 bits dans un cookie `__Host-session` (HttpOnly, Secure, SameSite=Strict). Seule l'empreinte SHA-256 est en base. Durée de 7 jours glissants, 30 jours maximum. La déconnexion et la désactivation du compte suppriment les sessions
Anti force brute	Connexion : 5 échecs par pseudo ou 20 par IP en 15 minutes, puis 429. Inscription : 10 échecs par IP et par heure. Élévation : 5 échecs en 15 minutes
Énumération	Même message et même temps de calcul (hash factice) pour un pseudo inconnu ou un mauvais mot de passe
CSRF	SameSite=Strict, en-tête `Origin` obligatoire et identique au site pour POST, PUT, PATCH et DELETE, et JSON obligatoire
Entrées	Zod en mode strict : les champs inconnus sont refusés (par exemple, on ne peut pas envoyer `role: "admin"`). Corps limité à 16 Ko
SQL	Uniquement `.prepare("… ? …").bind()`. La règle ESLint `no-restricted-syntax` fait échouer la CI sinon, et elle est elle-même testée
Journal	Table `journal` : inscriptions, connexions réussies, échouées et bloquées, élévations, actions admin

3. Données de cartes (analysées le 2026-11-08, release v2026-11-04)

1407 cartes. Raretés présentes dans les données : Common, Uncommon, Rare, Epic, Showcase.

Set	Code	Ouvrable en booster
Origins	ogn	oui
Proving Grounds	ogs	non (produit starter)
Spiritforged	sfd	oui
Unleashed	unl	oui
Vendetta	ven	oui
Radiance	rad	oui

On classe les variantes à partir de `cardNumber` :

- `NNN-TTT` avec NNN ≤ TTT : carte de base, avec sa rareté.
- `NNNa-TTT` : Alt Art (Showcase).
- `NNN-TTT` avec NNN > TTT : Overnumbered (Showcase).
- `NNN-star-TTT` : Overnumbered signée (Showcase).
- `spX-NNN` : Special Alt (Showcase).
- `rNN` et `rNNa` : Rune et Rune alt. `tNN` : Token ou Battlefield token.

3bis. Catalogue (étape 4)

- **Source :** le miroir Hugging Face officiel de l'auteur `https://huggingface.co/datasets/Wysme/riftbound-cards/resolve/main/cards.json` en premier, avec la release GitHub `…/releases/latest/download/cards.json` en secours. C'est le même fichier d'environ 1,5 Mo, mis à jour chaque jour par l'auteur. Chaque source a 8 s au plus. Si les deux échouent et que rien n'est en cache, l'API répond 503 `catalogue_indisponible`. Rien n'est stocké en base.
- **Cache :** mémoire du Worker, puis Cache API Cloudflare (catalogue déjà transformé, 24 h), puis téléchargement. Si la source tombe en panne, on garde l'ancienne version en mémoire. Un seul téléchargement est lancé même si plusieurs requêtes arrivent en même temps.
- **Validation :** chaque carte de la source est validée par Zod (format du code, rareté connue, longueurs maximales, image venant obligatoirement de `https://cmsassets.rgpub.io/`). Une carte invalide est ignorée et comptée, jamais affichée.
- **Variante** (calculée à partir de `cardNumber`, voir `serveur-api/catalogue/classification-cartes.ts`) : `base`, `alt` (NNNa), `overnumbered` (NNN > total), `signee` (NNN-star), `special` (spX), `ultime` (overnumbered de rareté Epic : Baron Nashor `unl-238-219`), `rune`, `rune_alt`, `token`.
- **Vérification du 2026-11-08 :** 1 407 cartes, 0 ignorée. Dans chaque set, le nombre de cartes `base` correspond au total imprimé (ogn 298, sfd 220, unl 219, ven 166, rad 167).
- **Attention pour l'étape 5 :** les runes d'Origins sont numérotées comme des cartes normales (variante `base`, `cardType` = `Rune`). Le slot Rune doit donc sélectionner sur `type === "Rune"`, pas seulement sur la variante. Spiritforged et Unleashed n'ont pas de rune dans la source, seulement des tokens.
- **Images :** le CDN Riot redimensionne. `?w=300&fm=webp` donne environ 28 Ko pour les miniatures, au lieu de 1,2 Mo pour le PNG d'origine, et `?w=744&fm=webp` sert pour la fiche.
- **API (connexion obligatoire) :** `GET /api/catalogue/sets`, `GET /api/catalogue/cartes?set=&rarete=&variante=&recherche=&page=` (60 par page, recherche sans accents ni majuscules), `GET /api/catalogue/cartes/:code`.
- **À surveiller :** le plan gratuit limite le CPU à 10 ms par requête. Le premier chargement (lecture du JSON de 1,5 Mo) peut dépasser cette limite. S'il y a des erreurs 1102 sur le catalogue en production, ajouter un stockage KV du catalogue transformé.

4. Composition d'un booster (14 cartes, source officielle Riot)

Slot	Nb	Contenu
Common	7	Common
Uncommon	3	Uncommon
Foil	1	Common, Uncommon, Rare ou Epic en foil
Rare+	2	Rare, Epic, Alt Art, Special Alt, Overnumbered, Signée
Rune / Token	1	Rune, Rune alt ou Token

Taux de référence, validés par l'utilisateur le 2026-11-08 (ils font foi, à coder dans un fichier de configuration à l'étape 5) :

Rareté	Fréquence par booster	Soit par display de 24
Epic (orange)	1 booster sur 6	4
Alt Art	1 booster sur 12	2
Overnumbered	1 booster sur 72	1 pour 3 displays
Overnumbered signée	1 booster sur 240	1 pour 10 displays

On les convertit en probabilités pour chacun des 2 slots Rare+. Comme il y a 2 slots, « 1 pour N boosters » devient 1/(2N) par slot. Pour l'Epic, on retire d'abord la part du slot Foil (1 %), pour que le total observé reste de 1 sur 6 :

Résultat	Proba par slot Rare+	Calcul
Epic	7,83 %	(1/6 − 0,01) / 2
Alt Art	4,17 %	1/24
Overnumbered	0,694 %	1/144
Overnumbered signée	0,208 %	1/480
Special Alt (sp)	0,05 %	estimation, non fournie par l'utilisateur
Rare	87,05 %	le reste

Les tests de l'étape 5 simuleront au moins 100 000 boosters et vérifieront ces fréquences **par booster** : Epic environ 16,7 %, Alt environ 8,3 %, Over environ 1,39 %, signée environ 0,42 %.

Slot Foil (estimation communautaire) : 75 % Common, 20 % Uncommon, 4 % Rare, 1 % Epic.

Slot Rune : 92 % Rune, 8 % Rune alt. Si le set n'a pas de rune, on tire un Token du set ou une Rune d'Origins.

5. Démarche DevSecOps

- Le dépôt est **public** (choix du 2026-11-08). Sur le plan gratuit, c'est la seule façon d'avoir la protection de branche, CodeQL et la push protection. Le site reste privé grâce à l'authentification.
- Branche `main` protégée par un ruleset « Protection main » : PR obligatoire (0 approbation, on est seul), pas de force-push ni de suppression, merge en squash uniquement, et 4 contrôles requis : `Lint, types, tests, build`, `Audit des dépendances`, `CodeQL (SAST)`, `Gitleaks (secrets)`.
- `.github/workflows/verifications-qualite.yml` lance le lint (ESLint), le typecheck (tsc), les tests (Vitest), le build, `npm audit` et la dependency review sur les PR. Il vérifie aussi que `serveur-api/types-cloudflare-generes.d.ts` est à jour.
- `.github/workflows/analyses-securite.yml` lance CodeQL (`security-extended`, JS/TS et Actions) et Gitleaks, à chaque PR, à chaque push et chaque lundi.

- Dependabot (npm et Actions, chaque semaine), alertes et correctifs automatiques activés. Secret scanning et push protection activés. Signalement privé de vulnérabilités activé (voir `.github/SECURITY.md`).
- Chaîne d'approvisionnement : versions npm exactes (`.npmrc save-exact`), lockfile, `npm ci`, actions épinglées par SHA, `permissions` minimales.
- Override `minimatch@11.0.5` dans `package.json` : corrige GHSA-g3hv-75wg-m249 (ReDoS, moderate), remonté via eslint 10. On pourra le retirer quand eslint le corrigera.
- Les secrets (token Cloudflare) sont uniquement dans GitHub Secrets. Jamais dans le code.
- En-têtes de sécurité : CSP stricte, HSTS, X-Frame-Options DENY, Referrer-Policy.
- Rate limiting sur la connexion. Validation des entrées avec Zod. Requêtes SQL paramétrées uniquement.
- Le déploiement en production se fait automatiquement au merge sur `main`, via `.github/workflows/deploiement-production.yml`. Le workflow lance les tests, le build, les migrations D1 `--remote`, puis `wrangler deploy` et vérifie `/api/health`. Il utilise l'environnement GitHub `production`, limité à la branche `main`. Le jeton Cloudflare n'a que les droits Workers et D1.
- `/api/health` vérifie aussi la base (`SELECT 1`). Si la base est en panne, il répond 503 sans détail interne.
- Dependabot ignore les versions majeures de TypeScript : TS 7 n'est pas supporté par typescript-eslint (PR #2 fermée).

6. Feuille de route

#	Étape	Statut
0	Cadrage, architecture, analyse des données, système de rapports	Terminé
1	Outils (Git, Node), dépôt GitHub, squelette du monorepo, CI sécurité	Terminé
2	Compte Cloudflare, Worker et D1, premier déploiement "Hello" via la CI	Terminé
3	Authentification : invitations, connexion, sessions, limitation des tentatives. **Rôle admin** (compte de l'utilisateur uniquement), première page `/admin` pour gérer les invitations et les comptes. Règle ESLint qui interdit le SQL non préparé	Terminé
4	Catalogue des cartes (fetch et cache du JSON, API sets et cartes)	Terminé
5	Moteur de boosters (taux, tirage, tests statistiques) et timer 10 min / 15 max	À faire
6	Frontend : ouverture de booster animée, collection, filtres	À faire
7	Échanges entre joueurs	À faire
8	**Admin : monitoring en quasi temps réel** (voir §2bis)	À faire
9	PWA (installable, hors-ligne partiel), responsive	À faire
10	Durcissement sécurité (revue, en-têtes, tests d'abus) et mise en prod	À faire

7. Actions manuelles en attente (utilisateur)

1. Sur le portable : installer Git, Node.js LTS et GitHub CLI (`winget install --id Git.Git -e`, `OpenJS.NodeJS.LTS`, `GitHub.cli`), puis lancer `gh auth login`.

8. Journal

- **2026-11-08 — Étape 0.** Analyse de l'API (1407 cartes, 6 sets), recherche des taux officiels, choix de l'architecture, mise en place des rapports PDF (`outils/generer-rapport-pdf.py`).
- **2026-11-08 — Étape 1.** Dépôt public `Crousti-Bats/riftbound-tcg-prive` créé. Squelette : Worker Hono (`worker/`, route `/api/health`, en-têtes de sécurité, 3 tests), front React 19 et Vite 8 (`src/`), `@cloudflare/vite-plugin`, TypeScript strict, ESLint. CI et sécurité (CodeQL, Gitleaks, audit, dependency review, Dependabot), ruleset sur `main`. Le lint, les types, les tests et le build passent en local, avec 0 vulnérabilité.
- **2026-11-08 — Réorganisation.** Les fichiers sont rangés dans des dossiers par fonction, avec des noms explicites en français (voir ci-dessous). Les anciens chemins (`worker/`, `src/`, `docs/`, `tools/`) n'existent plus. Les rapports PDF 00 et 01 citent encore ces anciens chemins, c'est normal : ce sont des archives.
- **2026-11-08 — Étape 2.** `wrangler login` effectué sur le PC fixe. Base D1 `riftbound-tcg` créée (WEUR), migration `0001_initialisation.sql` (table `parametres`) appliquée en local et en production. Premier déploiement manuel réussi : https://riftbound-tcg.axelgirard.workers.dev/api/health répond `{"status":"ok","base":"ok"}`, avec les en-têtes CSP, HSTS et X-Frame-Options vérifiés. Workflow de déploiement automatique ajouté, environnement GitHub `production` créé (branche `main` uniquement). L'assistant n'a pas le droit d'écrire des secrets GitHub ni de fermer des PR : ces actions reviennent à l'utilisateur (§7). 4 tests passent.
- **2026-11-08 — Admin.** L'utilisateur demande une page admin de monitoring en quasi temps réel, réservée à lui seul. Ajout du §2bis. La feuille de route passe à 11 étapes : rôle admin à l'étape 3, monitoring à l'étape 8. Étape 3 : ajout d'une règle ESLint contre le SQL non préparé.
- **2026-11-08 — Taux de drop.** L'utilisateur a fourni les taux de référence : Epic 1/6, Alt 1/12, Over 1/72, Over signée 1/240 boosters. Ils remplacent les estimations initiales (Epic à 13,4 % et signée à 0,07 % par slot, jugées trop élevée et trop faible). Voir §4.
- **2026-11-08 — Clôture de l'étape 2.** Jeton API Cloudflare créé par l'utilisateur (modèle Edit Cloudflare Workers + D1, expire dans 1 an, **à renouveler vers le 2027-11-08**) et saisi dans l'environnement GitHub `production`. PR #2 fermée. Premier run « Déploiement production » réussi (tests, build, migrations, déploiement, vérification de /api/health).
- **2026-11-08 — Étape 3.** Authentification complète, détaillée au §2ter. 41 tests automatiques passent (inscription, connexion, sessions, CSRF, injection SQL, limitation, admin, règle ESLint). Parcours de bout en bout validé en local avec curl : invitation admin en ligne de commande, inscription, élévation, création d'invitation, journal, déconnexion. Vitest 5 est incompatible avec `@cloudflare/vitest-pool-workers` (qui demande vitest 4) : les tests utilisent `node:sqlite` avec les vraies migrations.
- **2026-11-08 — Correctif `admin:invitation`.** `wrangler d1 execute --remote --file` passe par l'API « import », qui refuse le jeton OAuth de `wrangler login` (Authentication error 10000). Le script utilise maintenant `--command`, qui passe par l'API « query », comme les migrations. Wrangler est lancé directement par Node, sans shell.
- **2026-11-08 — Clôture de l'étape 3.** Mot de passe minimum abaissé à 6 caractères (PR #8). Compte admin `Crousti` créé en production avec `npm run admin:invitation`, puis vérifié en base (role=admin, actif=1).
- **2026-11-08 — Étape 4.** Catalogue terminé (§3bis). 65 tests passent : classification, validation de données hostiles, cache, panne de la source, routes. Validé en local contre la vraie source : 1 407 cartes, premier chargement en 0,86 s, puis 0,05 s. Page Catalogue responsive avec filtres, miniatures WebP et fiche détaillée.
- **2026-11-08 — Correctif du catalogue vide en production.** Les journaux (`wrangler tail`) montraient les requêtes du catalogue bloquées 13,5 s sans CPU, puis annulées. Un Worker de diagnostic exécuté sur le réseau Cloudflare (`wrangler dev --remote`) a mesuré GitHub entre 3 et 9,5 s, contre 0,16 s pour Hugging Face. Le Cache API fonctionne bien sur workers.dev. Hugging Face devient la source principale, GitHub passe en secours, chaque source a 8 s au plus et la page affiche un message de chargement et d'erreur. 68 tests.

- Astuce diagnostic : `Date.now()` n'avance pas pendant le calcul dans un Worker (protection Spectre), donc on ne peut pas mesurer le CPU ainsi. Utiliser le champ `cpuTime` de `wrangler tail --format json`.
- Astuce : PowerShell `Invoke-WebRequest` refuse les cookies `Secure` sur http://localhost (les navigateurs les acceptent). Pour tester l'API en local, utiliser `curl` avec `-b/-c` dans Git Bash.
- Astuce Bash : les heredocs contenant des apostrophes françaises échouent dans l'outil Bash de l'assistant. Écrire plutôt les scripts dans un fichier.
- Astuce Windows : `pkill` n'existe pas dans Git Bash. Pour arrêter un serveur de dev resté ouvert, utiliser PowerShell `Get-CimInstance Win32_Process -Filter "Name='node.exe'"` puis `Stop-Process`.

Arborescence (convention de nommage : français, minuscules, tirets)

```
application-web/                        LE SITE (navigateur : téléphone, tablette, PC)
  index.html                            page HTML de départ (nom imposé par le web et Cloudflare)
  demarrage-application.tsx             démarre React
  racine-application.tsx                charge le compte connecté et choisit la page à afficher
  pages/page-connexion.tsx              formulaire de connexion
  pages/page-inscription.tsx            création de compte avec un code d'invitation (lien /
inscription#code=...)
  pages/page-accueil.tsx                accueil du joueur connecté
  pages/page-admin.tsx                  administration : invitations, comptes, journal
  pages/page-catalogue.tsx              catalogue : filtres, grille d'images, fiche de carte
  outils/libelles-cartes.ts             noms français des raretés et variantes, couleurs, URL d'images
redimensionnées
  outils/appels-api.ts                  appels au serveur et messages d'erreur en français
  outils/navigation.ts                  changement de page sans rechargement, format des dates
  styles-globaux.css                    couleurs et mise en page communes
  fichiers-statiques/_headers           en-têtes de sécurité du site (nom imposé par Cloudflare)

serveur-api/                            L'API (Cloudflare Worker, Hono)
  point-entree-api.ts                   assemble la sécurité (en-têtes, taille max, CSRF, session) et les
routes
  entetes-securite.ts                   en-têtes de sécurité des réponses de l'API
  types-application.ts                  types partagés (compte connecté, rôles)
  validation-donnees.ts                 règles de validation (Zod) de tout ce que le navigateur envoie
  lecture-requete.ts                    lecture et validation du corps JSON
  journal-evenements.ts                 écriture dans le journal de sécurité et d'admin
  authentification/sessions.ts          cookie de session, chargement du compte, exigerConnexion,
exigerAdmin
  authentification/routes-authentification.ts   /api/auth : inscription, connexion, déconnexion, moi
  administration/routes-administration.ts       /api/admin : élévation, invitations, comptes, journal
  securite/mots-de-passe.ts             hachage PBKDF2-SHA256 (100 000 itérations)
  securite/jetons-et-empreintes.ts      jetons aléatoires, SHA-256, comparaison en temps constant
  securite/limitation-tentatives.ts     anti force brute (compteurs dans D1)
  securite/protection-csrf.ts           refuse les requêtes venant d'un autre site
  catalogue/types-catalogue.ts          types Carte, Set, Variante, Rareté
  catalogue/classification-cartes.ts    validation de la source et classement en variantes
  catalogue/source-cartes.ts            téléchargement et cache (mémoire, Cache API) de la source
  catalogue/routes-catalogue.ts         /api/catalogue : sets, recherche de cartes, détail
  tests/                                tests automatiques (vraie base SQLite en mémoire + vraies
migrations)
    outils/base-d1-de-test.ts           imite D1 avec node:sqlite
    outils/client-api-de-test.ts        appelle l'API comme un navigateur
    donnees/cartes-echantillon.json     47 vraies cartes (une par set et par catégorie)
  types-cloudflare-generes.d.ts         GÉNÉRÉ par `npm run cf-typegen`, ne pas modifier à la main

base-de-donnees/                        BASE SQLITE CLOUDFLARE D1
  migrations/0001_initialisation.sql    table parametres
  migrations/0002_comptes_et_sessions.sql  comptes, sessions, invitations, limitations, journal
  (les migrations sont numérotées et jamais modifiées après le merge : on en ajoute une nouvelle)

configuration/                          RÉGLAGES DES OUTILS
  cloudflare-wrangler.jsonc             Cloudflare : nom du service, règles de routage, base D1
```

```
  vite-site-et-api.config.ts          Vite : `npm run dev` et `npm run build`
  tests-vitest.config.ts              Vitest : où sont les tests
  qualite-code-eslint.config.js       ESLint : règles de qualité et de sécurité du code
  typescript/tsconfig.json            TypeScript : fichier principal, qui renvoie aux 4 suivants
  typescript/tsconfig-application-web.json
  typescript/tsconfig-serveur-api.json
  typescript/tsconfig-configuration.json
  typescript/tsconfig-tests.json

documentation/                        REPRISE DU PROJET
  demande-initiale.md                 ta demande d'origine, mot pour mot
  etat-du-projet.md                   CE FICHIER : décisions, étape en cours, feuille de route
  rapports-avancement/*.pdf           un rapport PDF par étape

outils/
  generer-rapport-pdf.py              fabrique le PDF de fin d'étape
  creer-invitation-admin.mjs          SEUL moyen de créer un compte admin (`npm run admin:invitation`)

.github/                              GITHUB (emplacement imposé)
  workflows/verifications-qualite.yml lint, types, tests, build, audit à chaque PR
  workflows/analyses-securite.yml     CodeQL et Gitleaks à chaque PR et chaque lundi
  workflows/deploiement-production.yml  mise en ligne automatique après merge sur main
  dependabot.yml                      mises à jour automatiques des dépendances (nom imposé)
  SECURITY.md                         politique de sécurité (nom imposé)

À la racine (emplacement imposé par les outils) :
  package.json / package-lock.json    dépendances et commandes npm
  .npmrc                              réglages npm (versions exactes)
  .gitignore / .gitattributes         ce que Git ignore, fins de ligne
  README.md                           page d'accueil GitHub
  CLAUDE.md                           consignes lues par l'assistant à chaque nouvelle conversation
```